BINUS
HIGHER EDUCATION

PATTERN MINING I

SESSION 6
COMP6140001 – DATA MINING

SUBJECT MATTER EXPERT
D6667-Said Achmad

BINUS
HIGHER EDUCATION

LEARNING OBJECTIVE

LO2 : to apply various data mining techniques

BINUS
HIGHER EDUCATION

OUTLINE

- Basic concepts
- Frequent itemset mining methods : Apriori

BINUS
HIGHER EDUCATION

BASIC CONCEPTS

BINUS
HIGHER EDUCATION

BASIC CONCEPTS

Frequent pattern

- A pattern (a set of items, subsequences, substructures, etc.) that occurs frequently in a dataset
- An intrinsic and important property of datasets
- First proposed by Agrawal, Imielinski, and Swami [AIS93] in the context of frequent itemsets and association rule mining
- Motivation: finding inherent regularities in data
- Applications: basket data analysis, cross-marketing, catalog design, sale campaign analysis, Web log (click stream) analysis, and DNA sequence analysis.

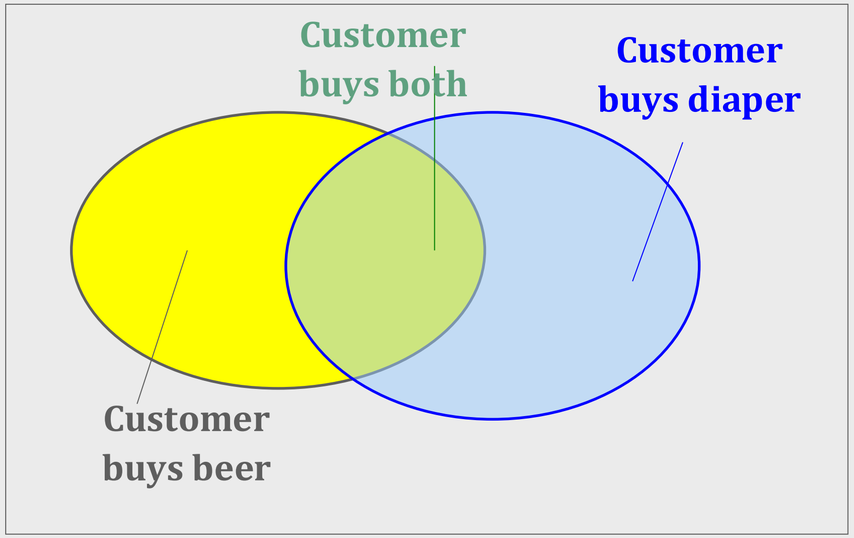
BASIC CONCEPTS (2)

- Foundation for many essential data mining tasks
  - Association, correlation, and causality analysis
  - Sequential, structural (e.g., sub-graph) patterns
  - Pattern analysis in spatiotemporal, multimedia, time-series, and stream data
  - Classification: discriminative, frequent pattern analysis
  - Cluster analysis: frequent pattern-based clustering
  - Data warehousing: iceberg cube and cube-gradient
  - Semantic data compression: fascicles
  - Broad applications An intrinsic and important property of datasets

BASIC CONCEPTS (3)

- Example:
  - itemset: a set of one or more items
  - k-itemset X = {x1, …, xk}
  - (absolute) support, or, support count of X: frequency or occurrence of an itemset X
  - (relative) support, s, is the fraction of transactions that contains X (i.e., the probability that a transaction contains X)
  - An itemset X is frequent if X's support is no less than a minsup threshold

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



BASIC CONCEPTS (4)

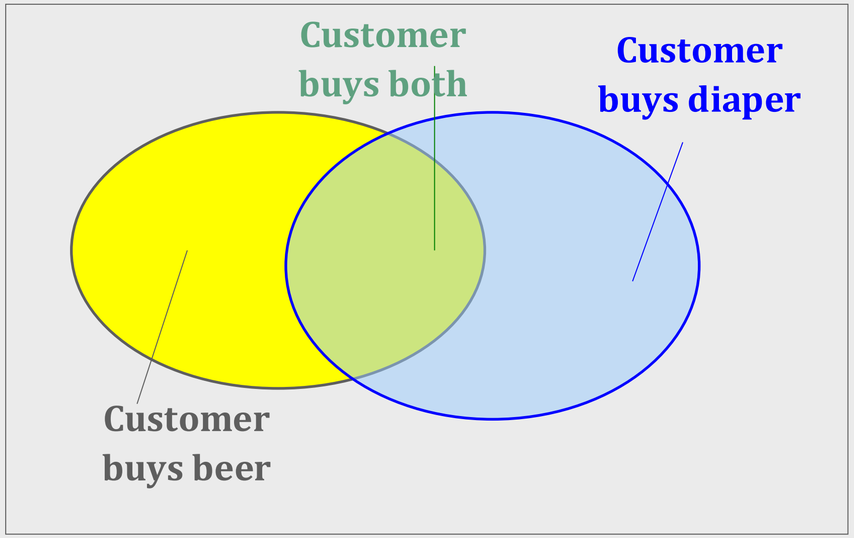
- Find all the rules X → Y with minimum support and confidence
  - support, s, probability that a transaction contains X ∪ Y
  - confidence, c, conditional probability that a transaction having X also contains Y

Let minsup = 50%, minconf = 50%

Freq. Pat.: Beer:3, Nuts:3, Diaper:4, Eggs:3, {Beer, Diaper}:3

- Association rules: (many more!)
  - Beer → Diaper (60%, 100%)
  - Diaper → Beer (60%, 75%)

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



BINUS
HIGHER EDUCATION

BASIC CONCEPTS (5)

Closed patterns and max-patterns

- A long pattern contains a combinatorial number of sub-patterns, e.g., $\{a_1, \dots, a_{100}\}$ contains $(100^1) + (100^2) + \cdots + (100^{100}) = 2^{100} - 1 = 1.27 * 10^{30}$ sub-patterns!
- Solution: *Mine closed patterns and max-patterns instead*
- An itemset X is closed if X is *frequent* and there exists *no super-pattern* Y ↄ X, *with the same support* as X (proposed by Pasquier, et al. @ ICDT'99)
- An itemset X is a max-pattern if X is frequent and there exists no frequent super-pattern Y ↄ X (proposed by Bayardo @ SIGMOD'98)
- Closed pattern is a lossless compression of freq. patterns
  - Reducing the # of patterns and rules

BASIC CONCEPTS (6)

- Example:

  DB = {<a_1, …, a_{100}>, < a_1, …, a_{50}>},

  Min_sup = 1

  Solutions:

  - Closed itemset:
    $< a_1, \dots, a_{100} > = 1$
    $< a_1, \dots, a_{50} > = 2$
  - Max-pattern
    $< a_1, \dots, a_{100} > = 1$
  - All patterns: !!

BINUS
HIGHER EDUCATION

BASIC CONCEPTS (7)

Computational complexity of frequent itemset mining

- How many itemsets are potentially to be generated in the worst case?
  - The number of frequent itemsets to be generated is sensitive to the minsup threshold
  - When minsup is low, there exist potentially an exponential number of frequent itemsets
  - The worst case: M^N where M: # distinct items, and N: max length of transactions
- The worst case complexity vs. the expected probability
  - Ex. Suppose Walmart has 10^4 kinds of products
    - The chance to pick up one product 10^{-4}
    - The chance to pick up a particular set of 10 products: $\sim 10^{-40}$
    - What is the chance this particular set of 10 products to be frequent 10^3 times in 10^9 transactions?

FREQUENT ITEMSET MINING METHODS : APRIORI

BINUS
HIGHER EDUCATION

FREQUENT ITEMSET MINING METHODS

Downward closure properties

- Any subset of a frequent itemset must be frequent
- If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
- i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}

Scalable mining methods

- Apriori (Agrawal & Srikant@VLDB'94)
- Freq. pattern growth (FPgrowth—Han, Pei & Yin @SIGMOD'00)
- Vertical data format approach (Charm—Zaki & Hsiao @SDM'02)

BINUS
HIGHER EDUCATION

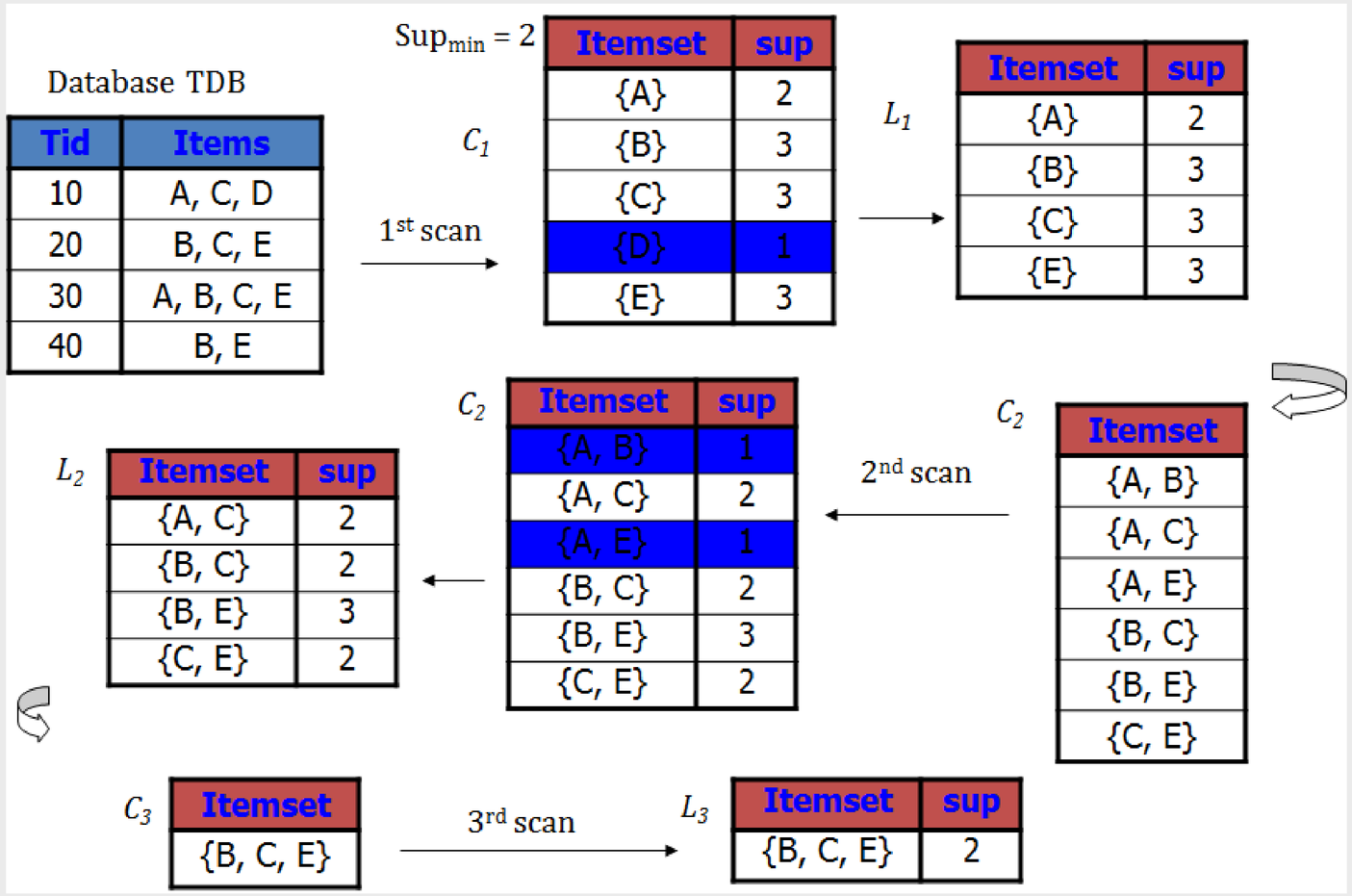
APRIORI ALGORITHM

A candidate generation and test approach

- Apriori pruning principle: if there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
  - Test the candidates against DB
  - Terminate when no frequent or candidate set can be generated

APRIORI ALGORITHM (2)

- Example:



BINUS
HIGHER EDUCATION

APRIORI ALGORITHM (3)

Pseudocode

C_k: Candidate itemset of size k
L_k : frequent itemset of size k

L_1 = {frequent items};
for ($k = 1; L_k! = \emptyset;$ k++) **do begin**
  C_{k+1} = candidates generated from L_k;
  for each transaction t in database do
    increment the count of all candidates in C_{k+1} that are contained in t
  L_{k+1} = candidates in C_{k+1} with min_support
  end
return $\cup_k L_k$;

APRIORI ALGORITHM (4)

Implementation

- Generate candidates
  - Step 1: self-joining L_k
  - Step 2: pruning
- Example of candidate-generation
  - L_3={*abc, abd, acd, ace, bcd*}
  - Self-joining: L_3*L_3
    - abcd from abc and abd
    - acde from acd and ace
- Pruning: acde is removed because ade is not in L_3
  - C_4 = {*abcd*}

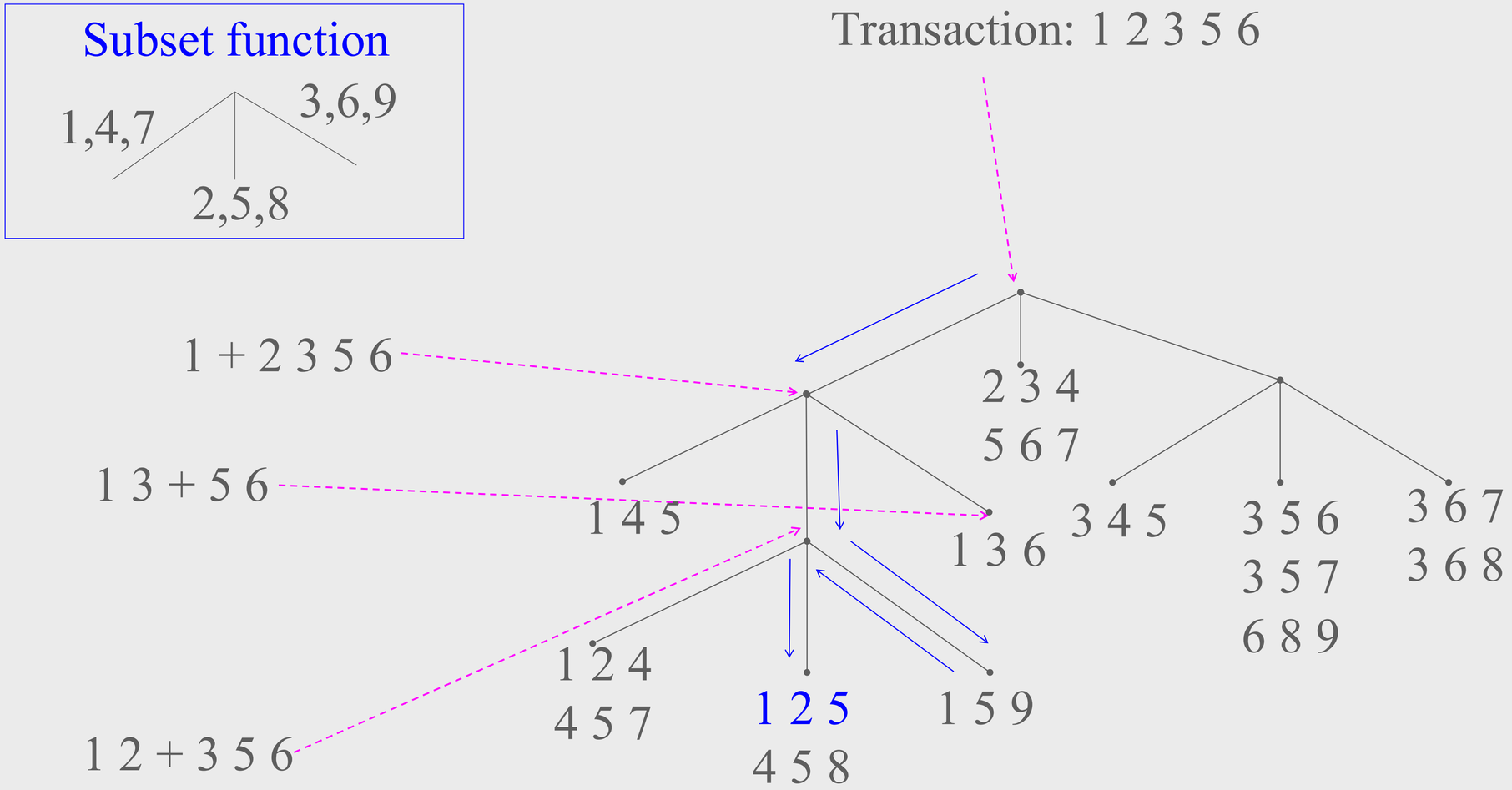
APRIORI ALGORITHM (5)

Count supports of candidates

- Why counting supports of candidates a problem?
  - The total number of candidates can be very huge
  - One transaction may contain many candidates
- Method:
  - Candidate itemsets are stored in a hash-tree
  - Leaf node of hash-tree contains a list of itemsets and counts
  - Interior node contains a hash table
  - Subset function: finds all the candidates contained in a transaction

APRIORI ALGORITHM (6)

Count supports of candidates using Hash Tree



APRIORI ALGORITHM (7)

Candidate generation: an SQL implementation

- SQL implementation of candidate generation
  - Suppose the items in L_{k-1} are listed in an order
  - Step 1: self-joining L_{k-1}

    insert into $\boldsymbol{C_k}$

    select $\boldsymbol{p.item_1, p.item_2, \ldots, p.item_{k-1}, q.item_{k-1}}$

    from $\boldsymbol{L_{k-1}\ p, L_{k-1}\ q}$

    where $\boldsymbol{p.item_1=q.item_1, \ldots, p.item_{k-2}=q.item_{k-2}, p.item_{k-1} < q.item_{k-1}}$
  - Step 2: pruning

    forall ***itemsets c in*** $\boldsymbol{C_k}$ do

      forall ***(k-1)-subsets s of c*** do

        if *(s is not in* L_{k-1}*)* **then delete** *c* **from** C_k
- Use object-relational extensions like UDFs, BLOBs, and Table functions for efficient implementation

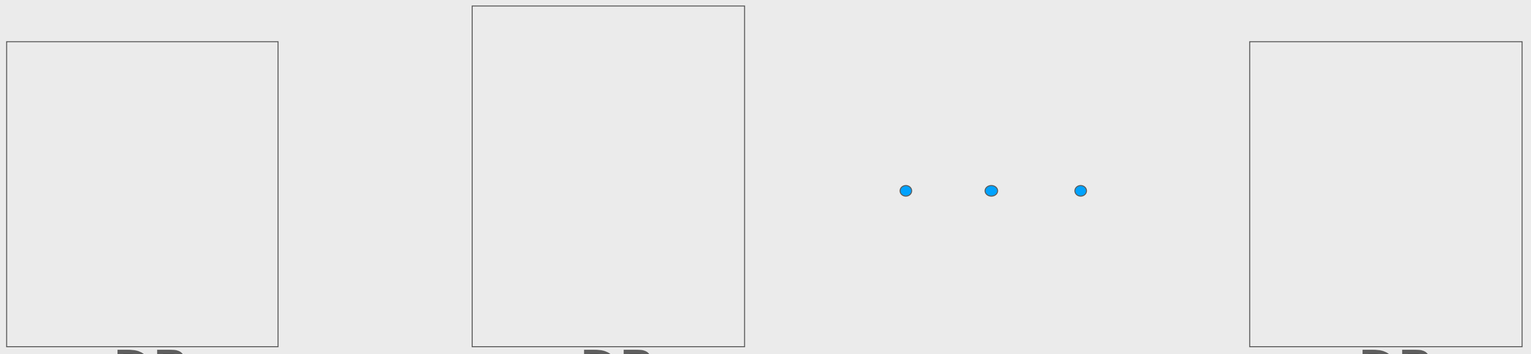
FURTHER IMPROVEMENT OF THE APRIORI METHOD

- Major computational challenges
  - Multiple scans of transaction database
  - Huge number of candidates
  - Tedious workload of support counting for candidates
- Improving Apriori: general ideas
  - Reduce passes of transaction database scans
  - Shrink number of candidates
  - Facilitate support counting of candidates

FURTHER IMPROVEMENT OF THE APRIORI METHOD (2)

Partition: scan database only twice

- Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB
  - Scan 1: partition database and find local frequent patterns
  - Scan 2: consolidate global frequent patterns
- A. Savasere, E. Omiecinski and S. Navathe, VLDB'95



FURTHER IMPROVEMENT OF THE APRIORI METHOD (3)

DHP: reduce the number of candidates

- A k-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent
  - Candidates: a, b, c, d, e
  - Hash entries
    - {ab, ad, ae}
    - {bd, be, de}
    - …
  - Frequent 1-itemset: a, b, d, e
  - ab is not a candidate 2-itemset if the sum of count of {ab, ad, ae} is below support threshold
- J. Park, M. Chen, and P. Yu. An effective hash-based algorithm for mining association rules. SIGMOD'95

Count	Itemsets
35	{ab, ad, ae}
88	{bd, be, de}
.	.
.	.
.	.
102	{yz, gs, wt}

Hash Table

FURTHER IMPROVEMENT OF THE APRIORI METHOD (4)

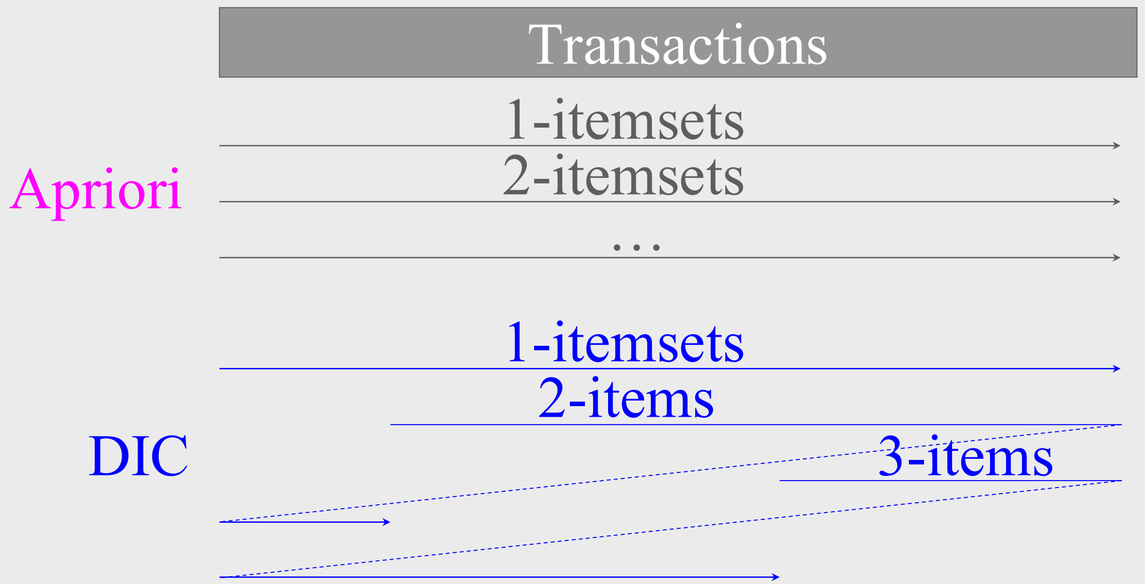
Sampling for frequent patterns

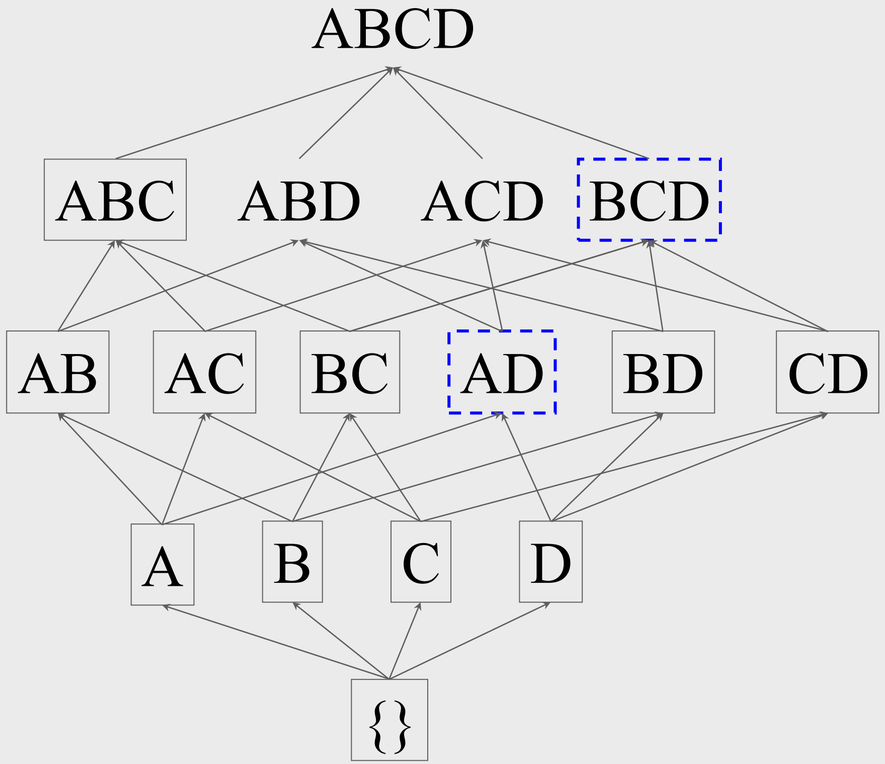
- Select a sample of original database, mine frequent patterns within sample using Apriori
- Scan database once to verify frequent itemsets found in sample, only borders of closure of frequent patterns are checked
    - Example: check abcd instead of ab, ac, …, etc.
- Scan database again to find missed frequent patterns
- H. Toivonen. Sampling large databases for association rules. In *VLDB'96*

FURTHER IMPROVEMENT OF THE APRIORI METHOD (5)

DIC: Reduce Number of Scans

- Once both A and D are determined frequent, the counting of AD begins
- Once all length-2 subsets of BCD are determined frequent, the counting of BCD begins





Itemset lattice

SUMMARY

- Basic concepts: association rules, support-confident framework, closed and max-patterns
- Scalable frequent pattern mining methods
    - Apriori (Candidate generation & test)

EXERCISE

1. Imagine you are analyzing customer purchases in a supermarket. Give one example of a useful pattern that could improve sales
2. Why is the 'minimum support threshold' important in the Apriori algorithm? What happens if the threshold is too high or too low?

REFERENCES

Han, J., Kamber, M., & Pei, J. (2023). Data Mining: Concepts and Techniques (4rd ed.). San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.